

CHAPTER 9

GRAPHS

Learning Outcomes

- ▶ At the end of this chapter, you should be able to
 - ▶ Describe what a graph is and its components
 - ▶ Provide examples of graph applications
 - ▶ Describe a graph's representations
 - ▶ Apply Depth-First and Breadth-First graph traversal algorithms
 - ▶ Represent a suitable problem as a graph and implement the graph in code along with its operations

Overview

- ▶ Basic Terminologies
- ▶ Representing Graphs
- ▶ Graph Traversals
 - ▶ Depth First Search (DFS)
 - ▶ Breadth First Search (BFS)

PART A

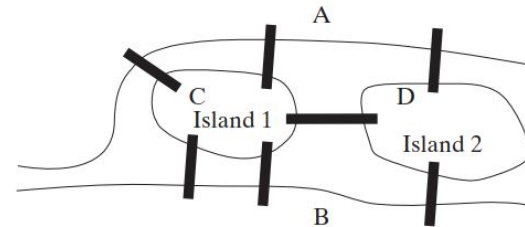
GRAPHS

Basic Terminologies

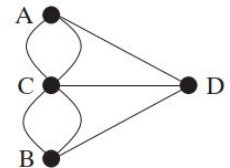
GRAPHS

Graph Theory

- ▶ The study of graph problems is known as graph theory.
- ▶ Graph theory was founded by Leonhard Euler in 1736, when he introduced graph terminology to solve the famous Seven Bridges of Königsberg problem.
- ▶ The city of Königsberg, Prussia (now Kaliningrad, Russia), was divided by the Pregel River.
- ▶ There were two islands on the river. The city and islands were connected by seven bridges.
- ▶ The question is,
 - ▶ can one take a walk,
 - ▶ cross each bridge exactly once,
 - ▶ and return to the starting point?
- ▶ Euler proved that it is not possible.



(a) Seven bridges sketch



(b) Graph model

Graphs

- ▶ Graphs provide the ultimate in data structure flexibility.
- ▶ Graphs can model both real-world systems and abstract problems.
- ▶ They are used in hundreds of applications.

Graph Applications

- ▶ Modeling connectivity in computer and communications networks.
- ▶ Representing a map as a set of locations with distances between locations; used to compute shortest routes between locations.
- ▶ Modeling flow capacities in transportation networks.
- ▶ Finding a path from a starting condition to a goal condition; for example, in artificial intelligence problem solving (e.g. robotics).

Graph Applications

- ▶ Modeling computer algorithms, showing transitions from one program state to another.
- ▶ Finding an acceptable order for finishing subtasks in a complex activity, such as constructing large buildings.
- ▶ Modeling relationships such as family trees, business or military organizations, and scientific taxonomies.

Tunku Abdul Rahman University College

Tunku Abdul Rahman University College

Add destination

Leave now

OPTIONS

Send directions to your phone

via Jalan Tun Hussien Onn/Route 3/A1

28 min

22.1 km

Fastest route, lighter traffic than usual

DETAILS

via Jalan Datuk Panglima Banting and Jalan Tun Hussien Onn/Route 3/A1

39 min

23.7 km

Lighter traffic than usual

via Jalan Bantayan Minintod

40 min

22.1 km

Lighter traffic than usual

Explore Tunku Abdul Rahman University College

P

The map displays three travel routes between Tunku Abdul Rahman University College and Kota Kinabalu International Airport. The fastest route (28 min, 22.1 km) is highlighted in blue. Other routes are shown in grey and orange. The map includes labels for various locations such as Sepanggar Island, Gaya Island, Jesselton Point, Sabah, Tanjong Aru, Petagas, Putatan, Lok Kawi Wildlife Park, and Telipok. Road numbers 606, 3, 1, 500, and A2 are also visible. A satellite view inset is located in the bottom left corner of the map area.

Route Description	Estimated Time	Distance	Traffic Status
via Jalan Tun Hussien Onn/Route 3/A1	28 min	22.1 km	Fastest route, lighter traffic than usual
via Jalan Datuk Panglima Banting and Jalan Tun Hussien Onn/Route 3/A1	39 min	23.7 km	Lighter traffic than usual
via Jalan Bantayan Minintod	40 min	22.1 km	Lighter traffic than usual

What is a graph?

- ▶ A graph is a mathematical structure that represents relationships among entities in the real world.
- ▶ A graph consists of
 - ▶ a nonempty set of vertices (also known as nodes or points), and
 - ▶ a set of edges that connect the vertices.
- ▶ For convenience, we define a graph as

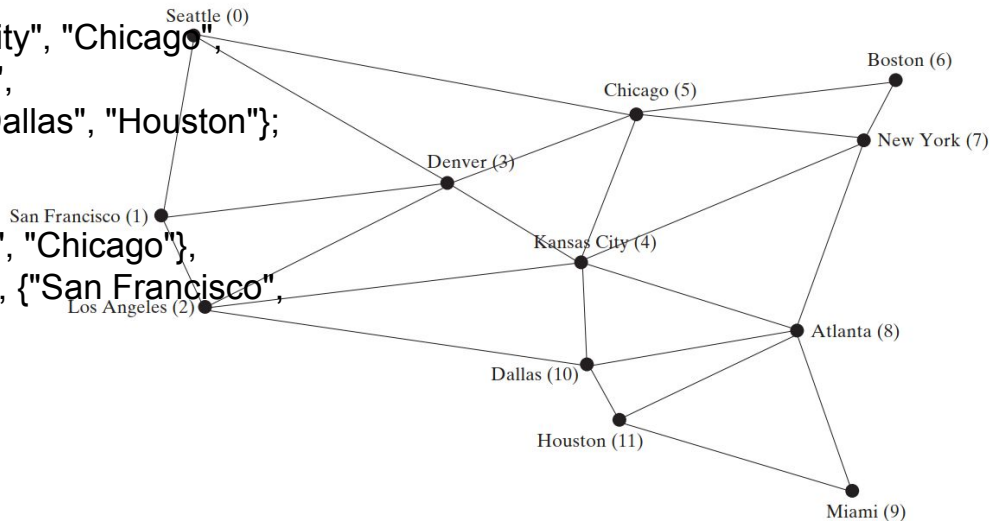
$$G = (V, E)$$

- ▶ where V represents a set of vertices and E represents a set of edges.

Flights Between US Cities

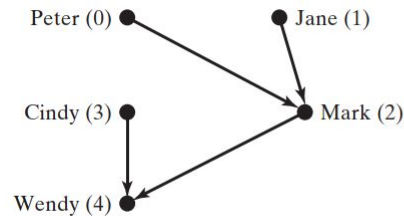
$V = \{\text{"Seattle", "San Francisco", "Los Angeles", "Denver", "Kansas City", "Chicago", "Boston", "New York", "Atlanta", "Miami", "Dallas", "Houston"}\};$

$E = \{\{\text{"Seattle", "San Francisco"}\}, \{\text{"Seattle", "Chicago"}\}, \{\text{"Seattle", "Denver"}\}, \{\text{"San Francisco", "Denver"}\}, \dots\};$

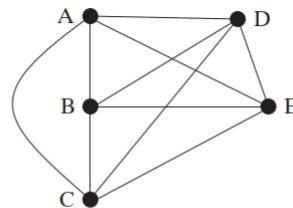


Graphs may appear in many forms

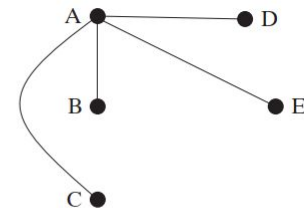
- ▶ A graph may be **directed** or **undirected**.
- ▶ In a directed graph, each edge has a direction, which indicates that you can move from one vertex to the other through the edge.



(a) A directed graph



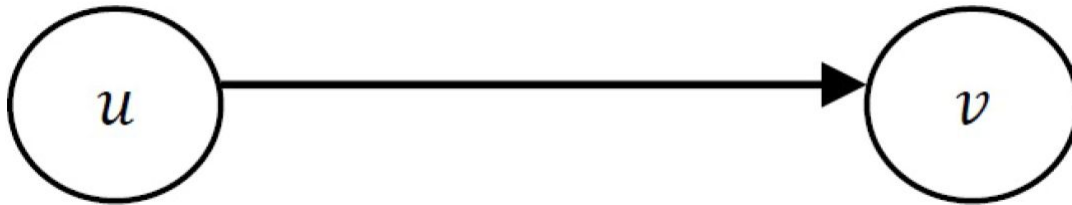
(b) A complete graph



(c) A subgraph of the graph in (b)

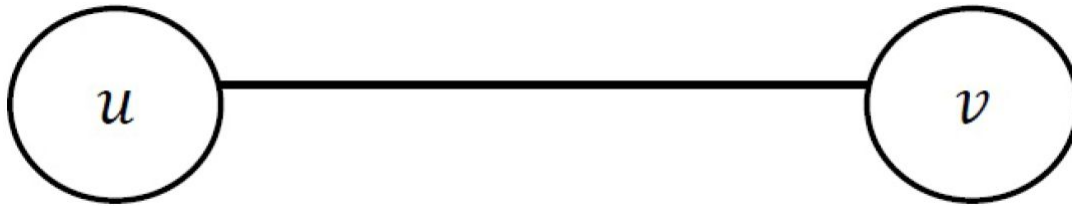
Directed Edge

- ▶ Ordered pair of vertices (u, v)
- ▶ First vertex u is the origin
- ▶ Second vertex v is the destination
- ▶ Example: one-way road traffic



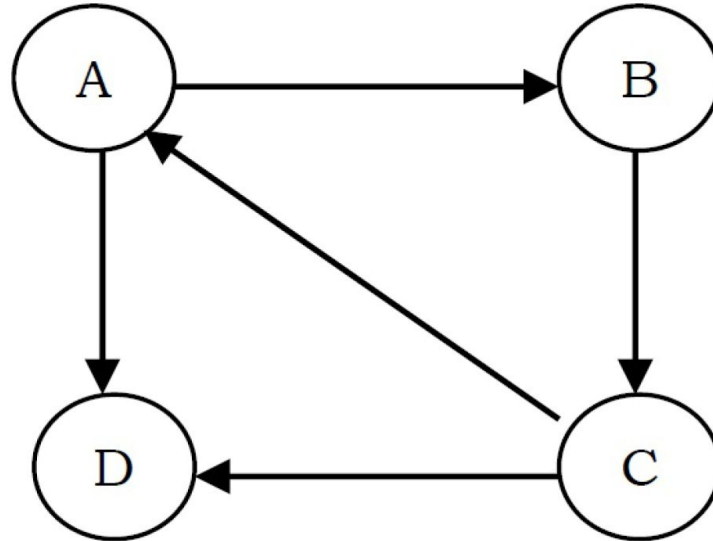
Undirected Edge

- ▶ unordered pair of vertices (u, v)
- ▶ Example: railway lines



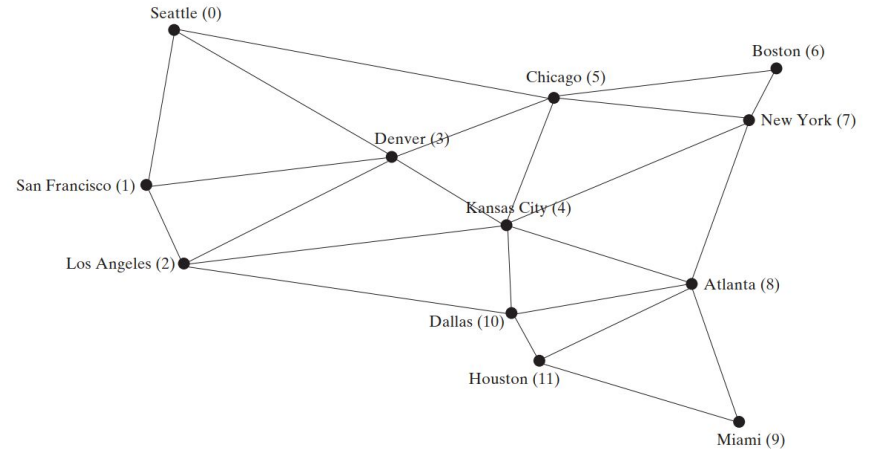
Directed Graph

- ▶ All the edges are directed
- ▶ Example: route network¹



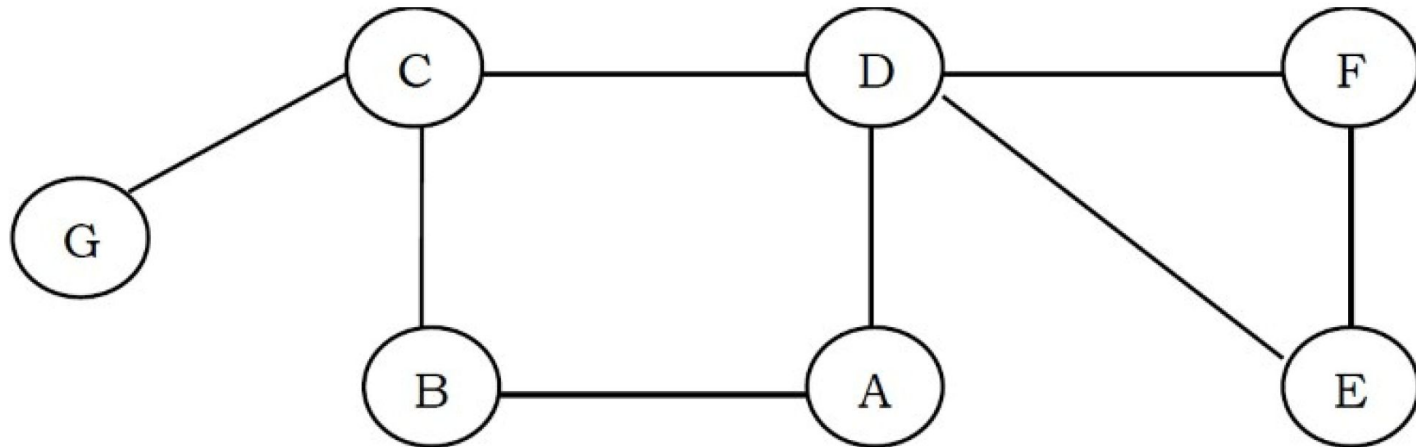
Undirected Graph

- ▶ All the edges are undirected
- ▶ Example: flight network



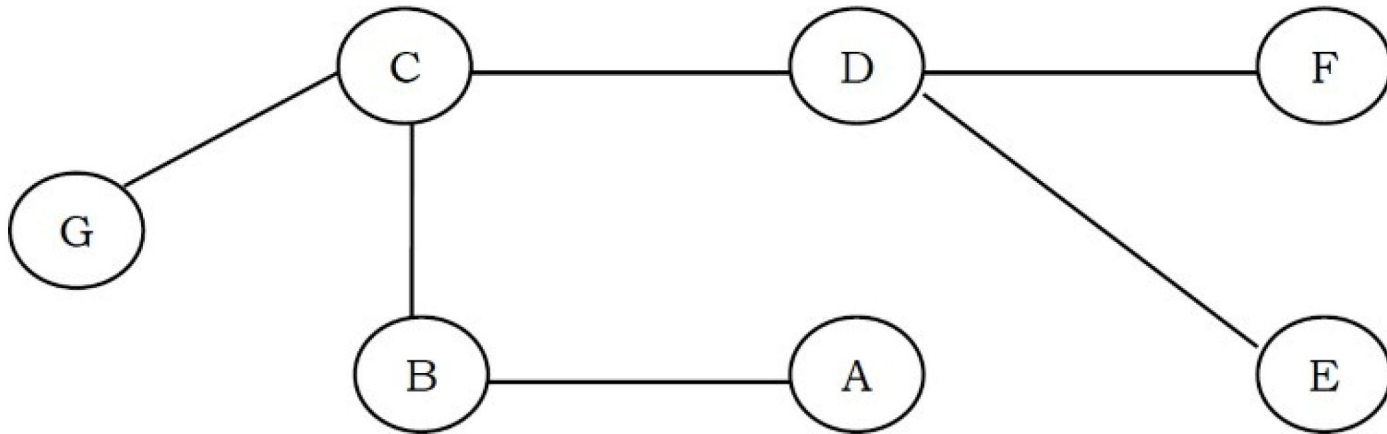
Adjacency and Incident

- ▶ When an edge connects two vertices, the vertices are said to be adjacent to each other and the edge is incident on both vertices.



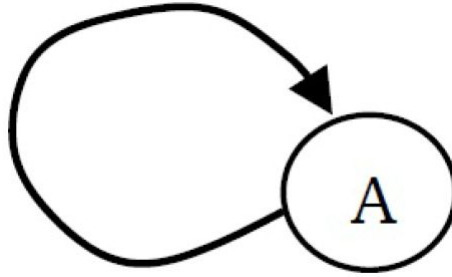
Tree (Acyclic Connected Graph)

- ▶ A graph with no cycles is called a tree.
- ▶ A tree is an acyclic connected graph.



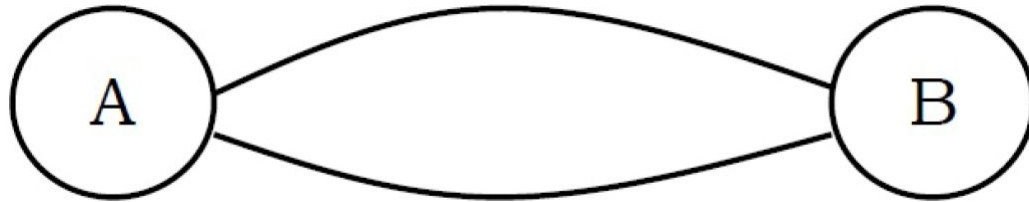
Self Looping Edge

- ▶ A self loop is an edge that connects a vertex to itself.



Parallel Edges

- ▶ Two edges are parallel if they connect the same pair of vertices.

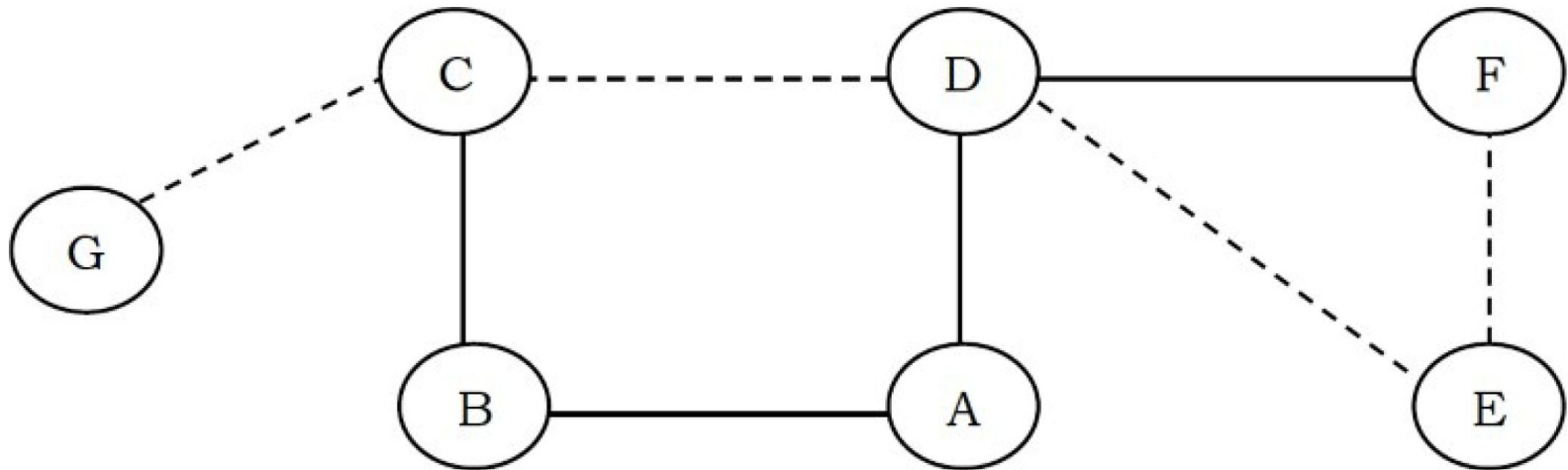


Degree of a vertex

- ▶ The Degree of a vertex is the number of edges incident on it.
- ▶ A subgraph is a subset of a graph's edges (with associated vertices) that form a graph.
- ▶ A path in a graph is a sequence of adjacent vertices. Simple path is a path with no repeated vertices.

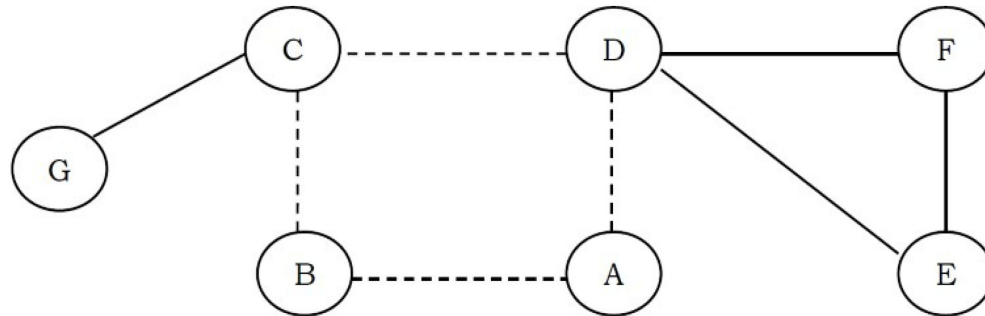
A Path in a Graph

- ▶ In the graph below, the dotted lines represent a path from G to E.

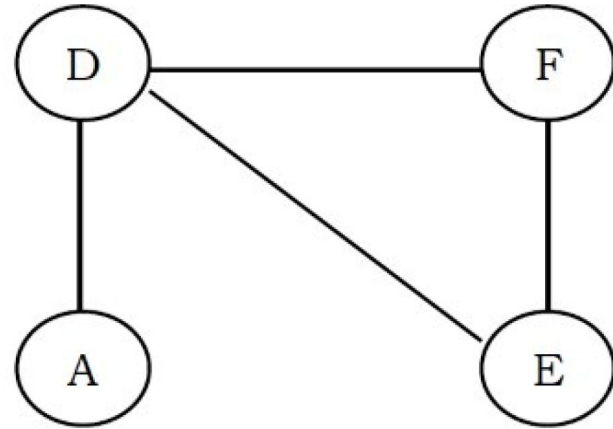
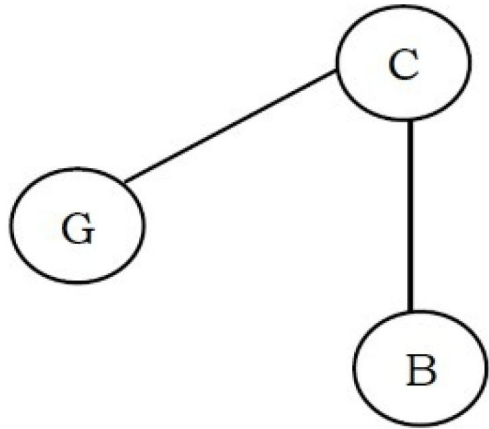


A Cycle in a Graph

- ▶ A cycle is a path where the first and last vertices are the same.
- ▶ A simple cycle is a cycle with no repeated vertices or edges (except the first and last vertices).

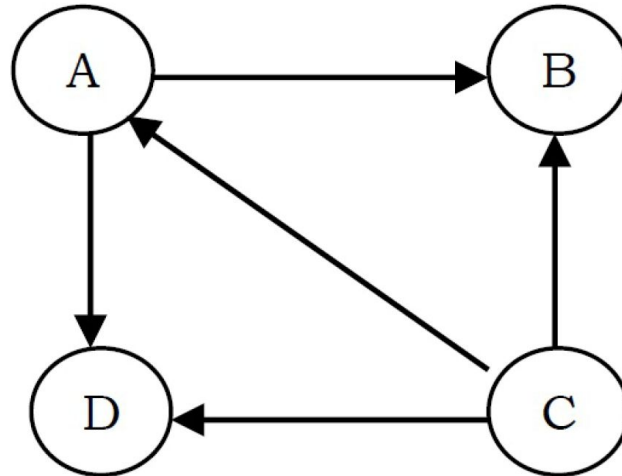


Connected Graph



Directed Acyclic Graph

- ▶ A directed acyclic graph (DAG) is a directed graph with no cycles.

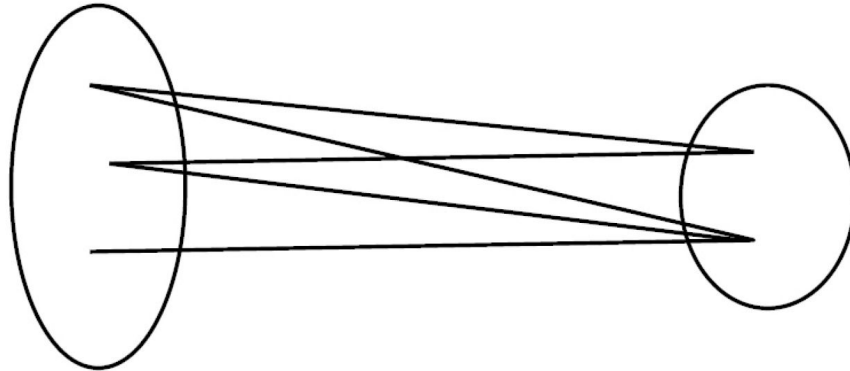


Forest and Spanning Tree

- ▶ A forest is a disjoint set of trees.
- ▶ A spanning tree of a connected graph is a subgraph that contains all of that graph's vertices and is a single tree.
- ▶ A spanning forest of a graph is the union of spanning trees of its connected components.

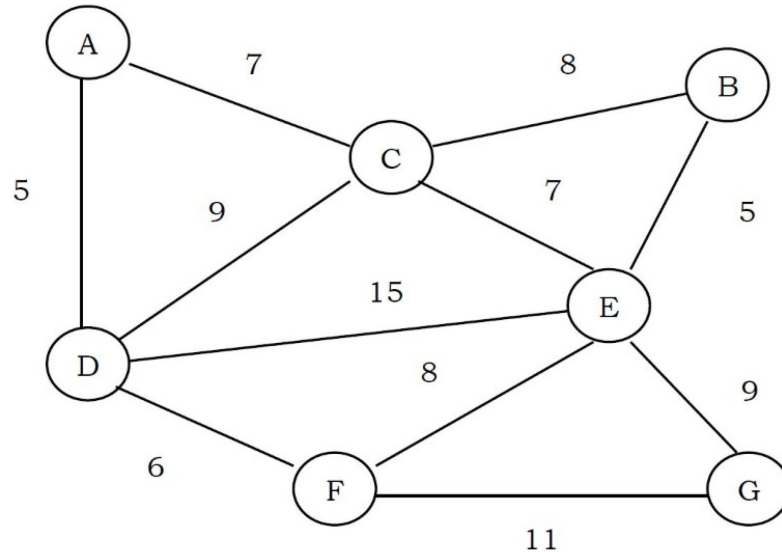
Bipartite Graph

- ▶ A bipartite graph is a graph whose vertices can be divided into two sets such that all edges connect a vertex in one set with a vertex in the other set.



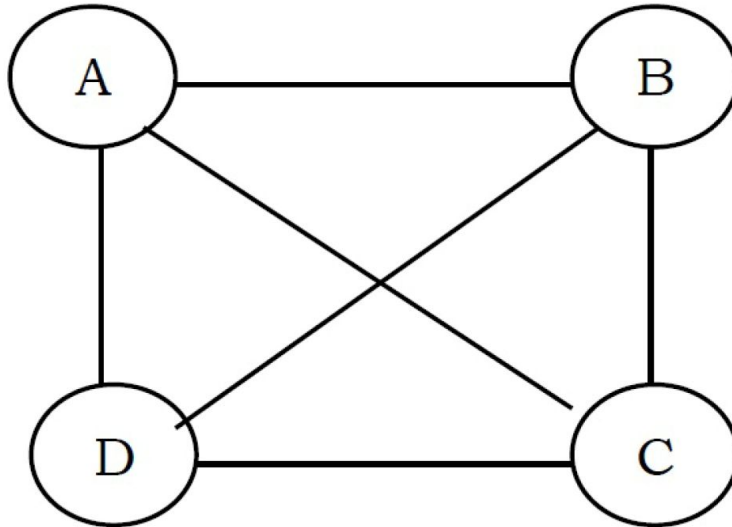
Weighted Graphs

- In weighted graphs integers (weights) are assigned to each edge to represent (distances or costs).



Complete Graphs

- ▶ Graphs with all edges present are called complete graphs.



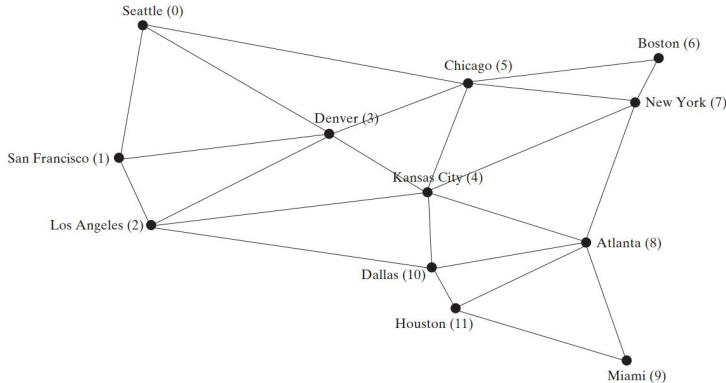
- ▶ Graphs with relatively few edges (generally if $\text{edges} < |V| \log |V|$) are called **sparse graphs**.
- ▶ Graphs with relatively few of the possible edges missing are called **dense**.
- ▶ Directed weighted graphs are sometimes called **network**.
- ▶ We will denote the number of vertices in a given graph by $|V|$, and the number of edges by $|E|$.
- ▶ Note that E can range anywhere from 0 to $|V|(|V| - 1)/2$ (in undirected graph). This is because each node can connect to every other node.

Representing Graphs

GRAPHS

Representing Vertices

- ▶ The vertices can be stored in an array or a list. For example, you can store all the city names in the graph below using the following array:



```
String[] vertices = {"Seattle", "San  
Francisco", "Los Angeles", "Denver",  
"Kansas City", "Chicago", "Boston", "New  
York", "Atlanta", "Miami", "Dallas",  
"Houston"};
```

An Array Stores the Vertex Names

- ▶ The vertices can be conveniently labeled using natural numbers $0, 1, 2, \dots, n-1$, for a graph for n vertices.
- ▶ Thus, `vertices[0]` represents "Seattle", `vertices[1]` represents "San Francisco", and so on...

<code>vertices[0]</code>	Seattle
<code>vertices[1]</code>	San Francisco
<code>vertices[2]</code>	Los Angeles
<code>vertices[3]</code>	Denver
<code>vertices[4]</code>	Kansas City
<code>vertices[5]</code>	Chicago
<code>vertices[6]</code>	Boston
<code>vertices[7]</code>	New York
<code>vertices[8]</code>	Atlanta
<code>vertices[9]</code>	Miami
<code>vertices[10]</code>	Dallas
<code>vertices[11]</code>	Houston

Representing Edges (Edge Array)

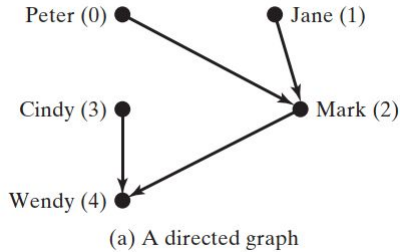
- ▶ The edges can be represented using a two-dimensional array.
- ▶ For example, you can store all the edges in the graph using the following array:

Representing Edges (Edge Array)

```
int[][] edges = {  
    {0, 1}, {0, 3}, {0, 5},  
    {1, 0}, {1, 2}, {1, 3},  
    {2, 1}, {2, 3}, {2, 4}, {2, 10},  
    {3, 0}, {3, 1}, {3, 2}, {3, 4}, {3, 5},  
    {4, 2}, {4, 3}, {4, 5}, {4, 7}, {4, 8}, {4, 10},  
    {5, 0}, {5, 3}, {5, 4}, {5, 6}, {5, 7},  
    {6, 5}, {6, 7},  
    {7, 4}, {7, 5}, {7, 6}, {7, 8},  
    {8, 4}, {8, 7}, {8, 9}, {8, 10}, {8, 11},  
    {9, 8}, {9, 11},  
    {10, 2}, {10, 4}, {10, 8}, {10, 11},  
    {11, 8}, {11, 9}, {11, 10}  
};
```

Representing Edges (Edge Array)

- ▶ This representation is known as the edge array. The vertices and edges can be represented as follows:



```
String[] names = {"Peter", "Jane", "Mark", "Cindy", "Wendy"};  
int[][] edges = {{0, 2}, {1, 2}, {2, 4}, {3, 4}};
```

Representing Edges (Edge Objects)

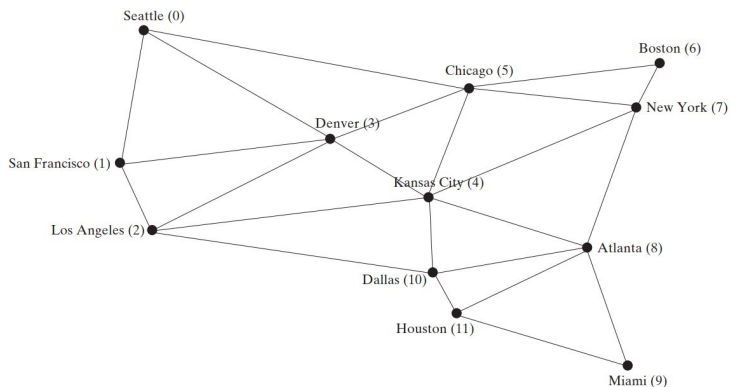
- ▶ Another way to represent the edges is to define edges as objects and store the edges in a `java.util.ArrayList`. The `Edge` class can be defined as follows:

Representing Edges (Edge Objects)

```
public class Edge {  
    int u;  
    int v;  
  
    public Edge(int u, int v) {  
        this.u = u;  
        this.v = v;  
    }  
  
    public boolean equals(Object o) {  
        return u == ((Edge)o).u && v == ((Edge)o).v;  
    }  
}
```

Representing Edges (Edge Objects)

- For example, you can store all the edges in the graph in Figure 28.1 using the following list



```
java.util.ArrayList<Edge> list = new
java.util.ArrayList<>();
list.add(new Edge(0, 1));
list.add(new Edge(0, 3));
list.add(new Edge(0, 5));
...
```


Representing Edges (Edge Objects)

- ▶ Storing Edge objects in an ArrayList is useful if you don't know the edges in advance.
- ▶ While representing edges using an edge array or Edge objects may be intuitive for input, it's not efficient for internal processing.
- ▶ In the next section, PART B introduces the representation of graphs using adjacency matrices and adjacency lists.
- ▶ These two data structures are efficient for processing graphs.



END OF PART A



GRAPHS

Part b

Graph Representation

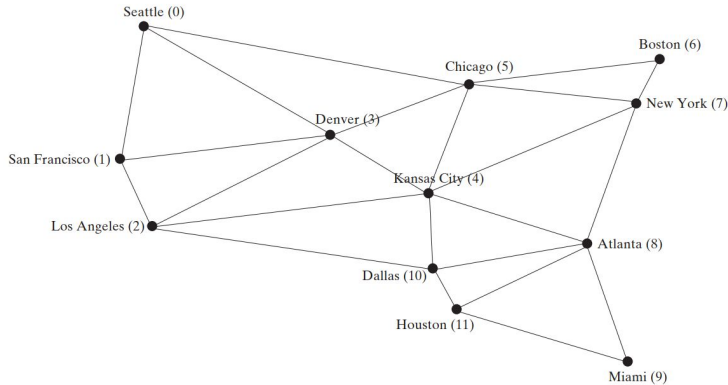
- ▶ To manipulate graphs we need to represent them in some useful form.
- ▶ There are two ways of doing this:
 - ▶ Adjacency Matrix
 - ▶ Adjacency List

Adjacency Matrices

- ▶ Assume that the graph has n vertices.
- ▶ You can use a two-dimensional $n * n$ matrix, say `adjacencyMatrix`, to represent the edges.
- ▶ Each element in the array is 0 or 1.
- ▶ `adjacencyMatrix[i][j]` is 1 if there is an edge from vertex i to vertex j ; otherwise,
- ▶ `adjacencyMatrix[i][j]` is 0. If the graph is undirected, the matrix is symmetric, because
- ▶ `adjacencyMatrix[i][j]` is the same as `adjacencyMatrix[j][i]`.

Adjacency Matrices

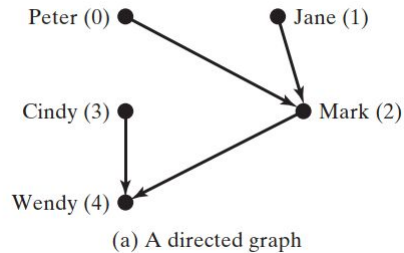
- Adjacency Matrix for undirected graph:



```
int[][] adjacencyMatrix = {
    {0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0}, // Seattle
    {1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0}, // San Francisco
    {0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0}, // Los Angeles
    {1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0}, // Denver
    {0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0}, // Kansas City
    {1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0}, // Chicago
    {0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0}, // Boston
    {0, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0}, // New York
    {0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 1}, // Atlanta
    {0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1}, // Miami
    {0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 1}, // Dallas
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1}, // Houston
};
```

Adjacency Matrices

- Adjacency Matrix for the directed graph:



```
int[][] a = {{0, 0, 1, 0, 0}, // Peter
              {0, 0, 1, 0, 0}, // Jane
              {0, 0, 0, 0, 1}, // Mark
              {0, 0, 0, 0, 1}, // Cindy
              {0, 0, 0, 0, 0}} // Wendy
};
```

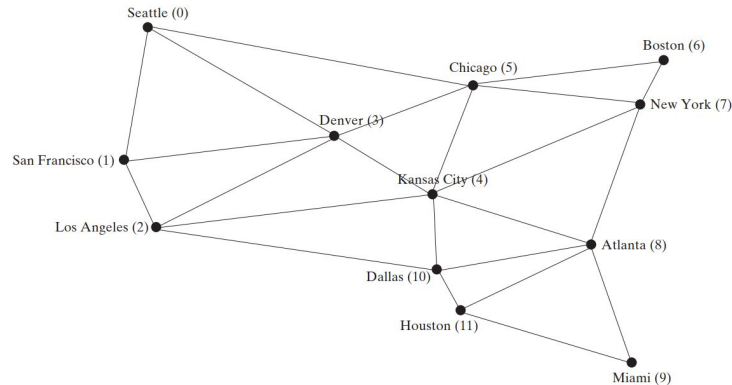
Adjacency Lists

- ▶ You can represent edges using adjacency vertex lists or adjacency edge lists.
- ▶ An adjacency vertex list for a vertex i contains the vertices that are adjacent to i and
- ▶ an adjacency edge list for a vertex i contains the edges that are adjacent to i .
- ▶ You may define an array of lists. The array has n entries, and each entry is a list.
- ▶ The adjacency vertex list for vertex i contains all the vertices j such that there is an edge from vertex i to vertex j .

Edges in the graph are represented using adjacency vertex lists

- For example, to represent the edges in the graph below you can create an array of lists as follows:

```
java.util.List<Integer>[] neighbors = new java.util.List[12];
```



Edges in the graph are represented using adjacency vertex lists

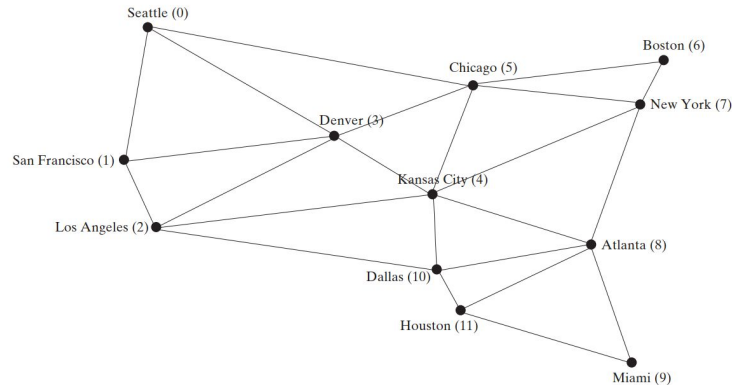
- neighbors[0] contains all vertices adjacent to vertex 0 (i.e., Seattle), neighbors[1] contains all vertices adjacent to vertex 1 (i.e., San Francisco), and so on.

Seattle	neighbors[0]	1	3	5								
San Francisco	neighbors[1]	0	2	3								
Los Angeles	neighbors[2]	1	3	4	10							
Denver	neighbors[3]	0	1	2	4	5						
Kansas City	neighbors[4]	2	3	5	7	8	10					
Chicago	neighbors[5]	0	3	4	6	7						
Boston	neighbors[6]	5	7									
New York	neighbors[7]	4	5	6	8							
Atlanta	neighbors[8]	4	7	9	10	11						
Miami	neighbors[9]	8	11									
Dallas	neighbors[10]	2	4	8	11							
Houston	neighbors[11]	8	9	10								

Edges in the graph are represented using adjacency edge lists

- To represent the adjacency edge lists for the graph below, you can create an array of lists as follows:

```
java.util.List<Edge>[] neighbors = new java.util.List[12];
```



Edges in the graph are represented using adjacency edge lists

- neighbors[0] contains all edges adjacent to vertex 0 (i.e., Seattle), neighbors[1] contains all edges adjacent to vertex 1 (i.e., San Francisco), and so on.

Seattle	neighbors[0]	Edge(0, 1)	Edge(0, 3)	Edge(0, 5)							
San Francisco	neighbors[1]	Edge(1, 0)	Edge(1, 2)	Edge(1, 3)							
Los Angeles	neighbors[2]	Edge(2, 1)	Edge(2, 3)	Edge(2, 4)	Edge(2, 10)						
Denver	neighbors[3]	Edge(3, 0)	Edge(3, 1)	Edge(3, 2)	Edge(3, 4)	Edge(3, 5)					
Kansas City	neighbors[4]	Edge(4, 2)	Edge(4, 3)	Edge(4, 5)	Edge(4, 7)	Edge(4, 8)	Edge(4, 10)				
Chicago	neighbors[5]	Edge(5, 0)	Edge(5, 3)	Edge(5, 4)	Edge(5, 6)	Edge(5, 7)					
Boston	neighbors[6]	Edge(6, 5)	Edge(6, 7)								
New York	neighbors[7]	Edge(7, 4)	Edge(7, 5)	Edge(7, 6)	Edge(7, 8)						
Atlanta	neighbors[8]	Edge(8, 4)	Edge(8, 7)	Edge(8, 9)	Edge(8, 10)	Edge(8, 11)					
Miami	neighbors[9]	Edge(9, 8)	Edge(9, 11)								
Dallas	neighbors[10]	Edge(10, 2)	Edge(10, 4)	Edge(10, 8)	Edge(10, 11)						
Houston	neighbors[11]	Edge(11, 8)	Edge(11, 9)	Edge(11, 10)							

Representing a Graph

- ▶ You can represent a graph using an adjacency matrix or adjacency lists.
- ▶ Which one is better?
- ▶ If the graph is dense (i.e., there are a lot of edges), using an adjacency matrix is preferred.
- ▶ If the graph is very sparse (i.e., very few edges), using adjacency lists is better, because using an adjacency matrix would waste a lot of space.

Efficiency of using Adjacency Matrices and Lists

- ▶ Both adjacency matrices and adjacency lists can be used in a program to make algorithms more efficient.
- ▶ For example, it takes $O(1)$ constant time to check whether two vertices are connected using an adjacency matrix, and it takes linear time $O(n)$ to print all edges in a graph using adjacency lists, where n is the number of edges.

Adjacency Vertex List vs. Adjacency Edge List

- ▶ Adjacency vertex list is simpler for representing unweighted graphs.
- ▶ However, adjacency edge lists are more flexible for a wide range of graph applications.
- ▶ It is easy to add additional constraints on edges using adjacency edge lists.

Representing Graphs using ArrayList

- ▶ You can use arrays, array lists, or linked lists to store adjacency lists.
- ▶ Using lists instead of arrays may be more suitable because the lists are easily expandable for adding new vertices.
- ▶ Using array lists instead of linked lists is preferred if it is only required to search for adjacent vertices in the list.

Representing Graph using an Adjacency Edge List

- Using array lists, the adjacency edge list c

Seattle	neighbors[0]	Edge(0, 1)	Edge(0, 3)	Edge(0, 5)			
San Francisco	neighbors[1]	Edge(1, 0)	Edge(1, 2)	Edge(1, 3)			
Los Angeles	neighbors[2]	Edge(2, 1)	Edge(2, 3)	Edge(2, 4)	Edge(2, 10)		
Denver	neighbors[3]	Edge(3, 0)	Edge(3, 1)	Edge(3, 2)	Edge(3, 4)	Edge(3, 5)	
Kansas City	neighbors[4]	Edge(4, 2)	Edge(4, 3)	Edge(4, 5)	Edge(4, 7)	Edge(4, 8)	Edge(4, 10)
Chicago	neighbors[5]	Edge(5, 0)	Edge(5, 3)	Edge(5, 4)	Edge(5, 6)	Edge(5, 7)	
Boston	neighbors[6]	Edge(6, 5)	Edge(6, 7)				
New York	neighbors[7]	Edge(7, 4)	Edge(7, 5)	Edge(7, 6)	Edge(7, 8)		
Atlanta	neighbors[8]	Edge(8, 4)	Edge(8, 7)	Edge(8, 9)	Edge(8, 10)	Edge(8, 11)	
Miami	neighbors[9]	Edge(9, 8)	Edge(9, 11)				
Dallas	neighbors[10]	Edge(10, 2)	Edge(10, 4)	Edge(10, 8)	Edge(10, 11)		
Houston	neighbors[11]	Edge(11, 8)	Edge(11, 9)	Edge(11, 10)			

```
List<ArrayList<Edge>> neighbors = new ArrayList<>();
neighbors.add(new ArrayList<Edge>());
neighbors.get(0).add(new Edge(0, 1));
neighbors.get(0).add(new Edge(0, 3));
neighbors.get(0).add(new Edge(0, 5));
neighbors.add(new ArrayList<Edge>());
neighbors.get(1).add(new Edge(1, 0));
neighbors.get(1).add(new Edge(1, 2));
neighbors.get(1).add(new Edge(1, 3));
...
...
neighbors.get(11).add(new Edge(11, 8));
neighbors.get(11).add(new Edge(11, 9));
neighbors.get(11).add(new Edge(11, 10));
```

Graph traversals

GRAPHS

Graph Traversal

- ▶ Graph traversal is the process of visiting each vertex in the graph exactly once.
- ▶ There are two popular ways to traverse a graph:
 - ▶ depth-first traversal (or depth-first search) and
 - ▶ breadth-first traversal (or breadth-first search).

Modelling Traversals

- ▶ Both traversals result in a spanning tree, which can be modeled using a class.
- ▶ The Traversal relation

AbstractGraph<V>.Tree

```
-root: int  
-parent: int[]  
-searchOrder: List<Integer>  
  
+Tree(root: int, parent: int[],  
      searchOrder: List<Integer>)  
+getRoot(): int  
+getSearchOrder(): List<Integer>  
+getParent(index: int): int  
+getNumberOfVerticesFound(): int  
+getPath(index: int): List<V>  
  
+printPath(index: int): void  
+printTree(): void
```

The root of the tree.

The parents of the vertices.

The orders for traversing the vertices.

Constructs a tree with the specified root, parent, and searchOrder.

Returns the root of the tree.

Returns the order of vertices searched.

Returns the parent for the specified vertex index.

Returns the number of vertices searched.

Returns a list of vertices from the specified vertex index to the root.

Displays a path from the root to the specified vertex.

Displays tree with the root and all edges.

Depth-First Search (DFS)

- ▶ In the case of a tree, the search starts from the root.
- ▶ In a graph, the search can start from any vertex.
- ▶ The depth-first search of a graph starts from a vertex in the graph and visits all vertices in the graph as far as possible before backtracking.
- ▶ A depth-first search of a tree first visits the root, then recursively visits the subtrees of the root.
- ▶ Similarly, the depth-first search of a graph first visits a vertex, then it recursively visits all the vertices adjacent to that vertex.

Depth-First Search (DFS)

- ▶ The difference is that the graph may contain cycles, which could lead to an infinite recursion.
- ▶ To avoid this problem, you need to track the vertices that have already been visited.
- ▶ The search is called depth-first because it searches “deeper” in the graph as much as possible.
- ▶ The search starts from some vertex v .
- ▶ After visiting v , it visits an unvisited neighbor of v . If v has no unvisited neighbor, the search backtracks to the vertex from which it reached v .
- ▶ We assume that the graph is connected and the search starting from any vertex can reach all the vertices.

Depth-First Search Algorithm

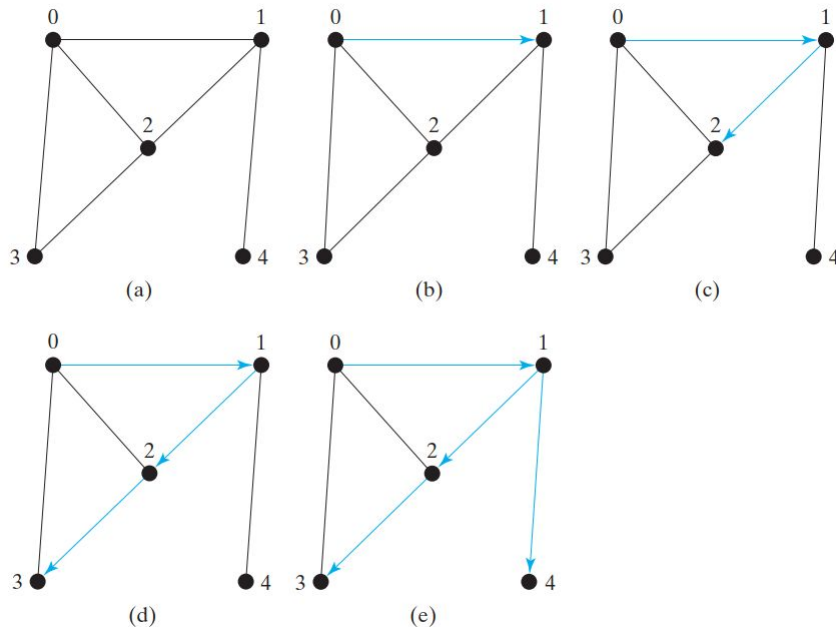
- The algorithm for the depth-first search is given below:

```
Input:  $G = (V, E)$  and a starting vertex  $v$   
Output: a DFS tree rooted at  $v$   
  
1 Tree dfs(vertex  $v$ ) {  
2   visit  $v$ ;  
3   for each neighbor  $w$  of  $v$   
4     if ( $w$  has not been visited) {  
5       set  $v$  as the parent for  $w$  in the tree;  
6       dfs( $w$ );  
7     }  
8 }
```

Depth-First Search Algorithm

- ▶ Depth-first search visits a node and its neighbors recursively.
- ▶ You can use an array named `isVisited` to denote whether a vertex has been visited.
- ▶ Initially, `isVisited[i]` is false for each vertex `i`. Once a vertex, say `v`, is visited, `isVisited[v]` is set to true.
- ▶ Consider the graph shown in (a). Suppose we start the depth-first search from vertex 0.
- ▶ First visit 0, then any of its neighbors, say 1. Now 1 is visited, as shown in (b).
- ▶ Vertex 1 has three neighbors—0, 2, and 4. Since 0 has already been visited, you will visit either 2 or 4.
- ▶ Let us pick 2. Now 2 is visited, as shown in (c).
- ▶ Vertex 2 has three neighbors: 0, 1, and 3. Since 0 and 1 have already been visited, pick 3.
- ▶ 3 is now visited, as shown in (d). At this point, the vertices have been visited in this order:

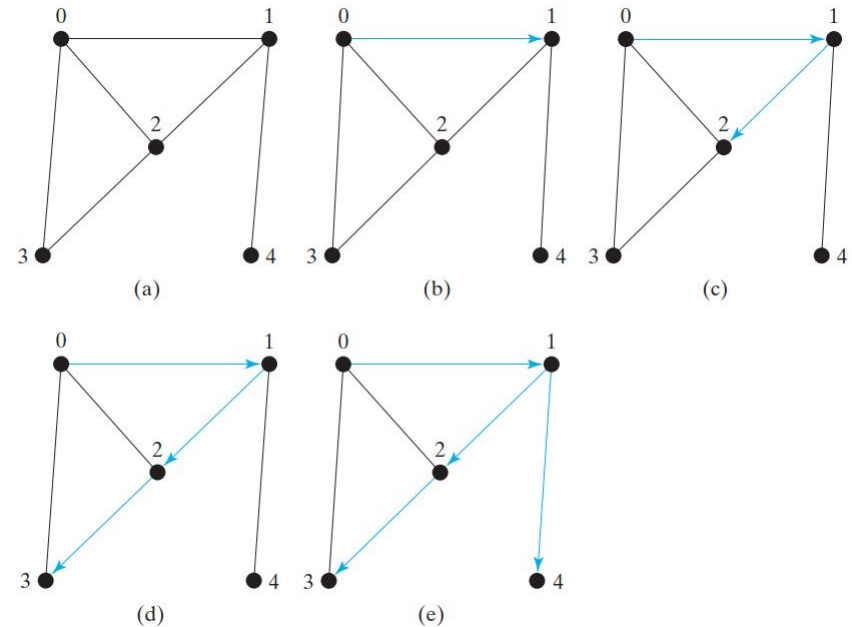
0, 1, 2, 3



Depth-First Search Algorithm

- ▶ Since all the neighbors of 3 have been visited, backtrack to 2.
- ▶ Since all the vertices of 2 have been visited, backtrack to 1. 4 is adjacent to 1, but 4 has not been visited.
- ▶ Therefore, visit 4, as shown in €. Since all the neighbors of 4 have been visited, backtrack to 1.
- ▶ Since all the neighbors of 1 have been visited, backtrack to 0. Since all the neighbors of 0 have
- ▶ been visited, the search ends.
- ▶ Since each edge and each vertex is visited only once, the time complexity of the dfs method is $O(|E| + |V|)$, where $|E|$ denotes the number of edges and $|V|$ the number of vertices.

0, 1, 2, 3, 4



Applications of the DFS

- ▶ The depth-first search can be used to solve many problems, such as the following:
 - ▶ Detecting whether a graph is connected. Search the graph starting from any vertex.
 - ▶ If the number of vertices searched is the same as the number of vertices in the graph, the graph is connected. Otherwise, the graph is not connected.

Applications of the DFS

- ▶ Detecting whether there is a path between two vertices
- ▶ Finding a path between two vertices
- ▶ Finding all connected components. A connected component is a maximal connected subgraph in which every pair of vertices are connected by a path
- ▶ Detecting whether there is a cycle in the graph
- ▶ Finding a cycle in the graph
- ▶ Finding a Hamiltonian path/cycle.
 - ▶ A Hamiltonian path in a graph is a path that visits each vertex in the graph exactly once. A Hamiltonian cycle visits each vertex in the graph exactly once and returns to the starting vertex

Breadth-First Search (BFS)

- ▶ The breadth-first search of a graph visits the vertices level by level.
- ▶ The first level consists of the starting vertex.
- ▶ Each next level consists of the vertices adjacent to the vertices in the preceding level.
- ▶ With breadth-first traversal of a tree, the nodes are visited level by level.
- ▶ First the root is visited, then all the children of the root, then the grandchildren of the root, and so on.
- ▶ Similarly, the breadth-first search of a graph first visits a vertex, then all its adjacent vertices, then all the vertices adjacent to those

Breadth-First Search Algorithm

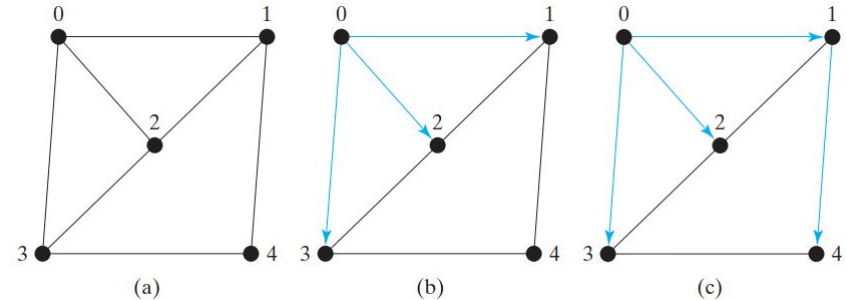
- ▶ The algorithm for the breadth first search starting from vertex v

```
Input:  $G = (V, E)$  and a starting vertex  $v$ 
Output: a BFS tree rooted at  $v$ 

1 Tree bfs(vertex  $v$ ) {
2   create an empty queue for storing vertices to be visited;
3   add  $v$  into the queue;
4   mark  $v$  visited;
5
6   while (the queue is not empty) {
7     dequeue a vertex, say  $u$ , from the queue;
8     add  $u$  into a list of traversed vertices;
9     for each neighbor  $w$  of  $u$ 
10      if  $w$  has not been visited {
11        add  $w$  into the queue;
12        set  $u$  as the parent for  $w$  in the tree;
13        mark  $w$  visited;
14      }
15   }
16 }
```

Breadth-First Search Algorithm

- ▶ Breadth-first search visits a node, then its neighbors, then its neighbors' neighbors, and so on.
- ▶ Consider the graph in Figure (a). Suppose you start the breadth-first search from vertex 0.
- ▶ First visit 0, then visit all its neighbors, 1, 2, and 3, as shown in (b).
- ▶ Vertex 1 has three neighbors: 0, 2, and 4. Since 0 and 2 have already been visited, you will now visit just 4, as shown in (c).
- ▶ Vertex 2 has three neighbors, 0, 1, and 3, which have all been visited. Vertex 3 has three neighbors, 0, 2, and 4, which have all been visited. Vertex 4 has two neighbors, 1 and 3, which have all been visited.
- ▶ Hence, the search ends.
- ▶ Since each edge and each vertex is visited only once, the time complexity of the bfs method is $O(|E| + |V|)$, where $|E|$ denotes the number of edges and $|V|$ the number of vertices.



Applications of the BFS

- ▶ Many of the problems solved by the DFS can also be solved using the BFS. Specifically, the BFS can be used to solve the following problems:
- ▶ Detecting whether a graph is connected.
 - ▶ A graph is connected if there is a path between any two vertices in the graph.
- ▶ Detecting whether there is a path between two vertices.
- ▶ Finding a shortest path between two vertices.
 - ▶ You can prove that the path between the root and any node in the BFS tree is a shortest path between the root and the node.

Applications of the BFS

- ▶ Finding all connected components.
 - ▶ A connected component is a maximal connected subgraph in which every pair of vertices are connected by a path.
- ▶ Detecting whether there is a cycle in the graph.
- ▶ Finding a cycle in the graph
- ▶ Testing whether a graph is bipartite.
 - ▶ (A graph is bipartite if the vertices of the graph can be divided into two disjoint sets such that no edges exist between vertices in the same set.)

Order of Node Visits:

- BFS: In BFS, the algorithm explores all the neighbors (adjacent nodes) of a node before moving on to the next level of neighbors. It visits nodes level by level, starting from the source node and moving outward.
- DFS: In DFS, the algorithm explores as far as possible along each branch before backtracking. It starts at the source node and explores as far as possible along each branch before backtracking to explore other branches.



END OF PART b



GRAPHS

END OF CHAPTER 9

GRAPHS